

Databricks Fundamentals

A hands-on introduction to the Databricks Lakehouse Platform — covering core concepts, data engineering with PySpark, Delta Lake, orchestration, and governance. Designed for data professionals ready to build production-grade pipelines.

Today's Agenda

01

Introduction

Goals, audience, training environment setup

03

Apache Spark Architecture

Driver, Executors, DAG, Lazy Evaluation, PySpark API

05

Delta Lake

ACID, CRUD, Time Travel, RESTORE, Schema Evolution

07

Unity Catalog & AI/BI

Governance, permissions, dashboards, Genie

02

Databricks Platform

Workspace, Compute, Unity Catalog, Delta Lake

04

Ingestion & Medallion Architecture

Bronze, Silver, Gold layers — transformations in PySpark

06

Orchestration & Workflows

Workflow Jobs, CRON scheduling, Repair Run

08

Final Project: RetailHub Pipeline

End-to-end pipeline built and scheduled as a job

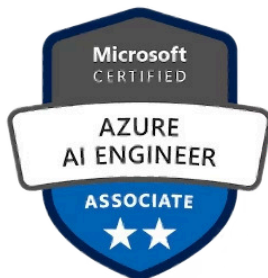
📅 Breaks: ~10 min every ~90 min · Lunch ~1:00 PM · Questions welcome at any time · Dedicated Q&A at the end of each block



Krzysztof Burejza

Data Engineer Azure

Data engineer with 10 years of experience working with data, from SQL Server to cloud platforms such as Azure, Databricks, and Microsoft Fabric. Focused on transforming raw data into valuable solutions, building data pipelines, data warehouses, and lakehouses. Enthusiast of new technologies at the intersection of data and artificial intelligence, which make data engineering even more fascinating.





Let's introduce ourselves!

Let's go around the meeting person by person – tell us a few words about yourself:

- Your name
- Your role or position
- Brief experience with the Databricks platform (if you have any)

Who Is This Training For?

Target Audience

- Data Analysts
- Data Engineers
- BI Specialists
- Technical Staff working with data pipelines

Prerequisites

- Basic SQL knowledge (SELECT, JOIN, GROUP BY)
- Familiarity with relational databases
- No Python experience required — we teach PySpark from scratch in context

What We Don't Assume

No prior Databricks, Spark, or cloud storage experience needed. We start from the fundamentals and build up progressively.



Training Rules

To ensure maximum effectiveness and comfort during our training, please familiarize yourself with the following rules:



Ask Anytime

Feel free to interrupt and ask questions. Your questions help everyone understand the material more deeply. There are no silly questions!



Labs are at Your Own Pace

You complete practical tasks at your own pace. If you finish early, check the "Bonus Challenges" section in the notebooks. Help others if they need support!



Mistakes are Welcome

Don't be afraid to make mistakes. It's the best way to learn, and that's what the **Time Travel** feature in Delta Lake, which we'll learn today, is for!



Collaborate

We encourage you to discuss and review solutions with your neighbor before running the code. Collaborative problem-solving deepens understanding.



Use AI Assistant

Remember the availability of the AI Assistant in Databricks. Use the shortcut `Cmd+i` (or `Ctrl+i` on Windows) to get help in the notebook, optimize code, or understand errors.

What You Will Learn Today

Platform

Navigating the Databricks environment, working with clusters, understanding the Lakehouse model

Spark

Understanding the execution model, writing transformations with the DataFrame API

Ingestion

Loading CSV/JSON files, building Bronze and Silver layers with explicit schemas

Delta Lake

ACID transactions, CRUD operations, Time Travel, and RESTORE

Orchestration

Creating Workflow Jobs, CRON scheduling, Repair Run for cost-efficient reruns

Final Project

A complete data pipeline scheduled as a production job — Bronze to Gold

How the Training Environment Works

→ Environment is Ready

Notebooks are pre-configured and the cluster is already running. No setup required — open your browser and you're in.

→ Two Languages, One Notebook

We write code in **PySpark** and **SQL** — both in the same notebook. Switch between them with magic commands.

→ Each Block: Theory → Hands-On Practice

We explain concepts on slides, then you immediately apply them in a notebook. Learning by doing is the core method.

→ Immediate Feedback

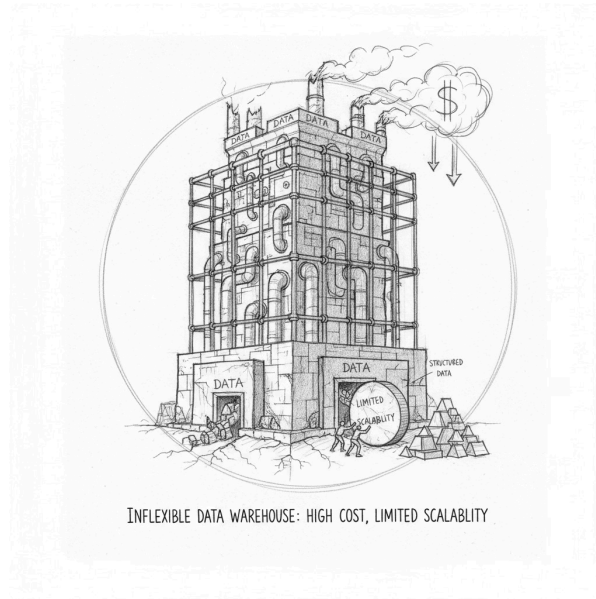
Results are visible right after running a cell. Errors appear inline — debugging is part of the learning experience.

SECTION 1

The Databricks Platform

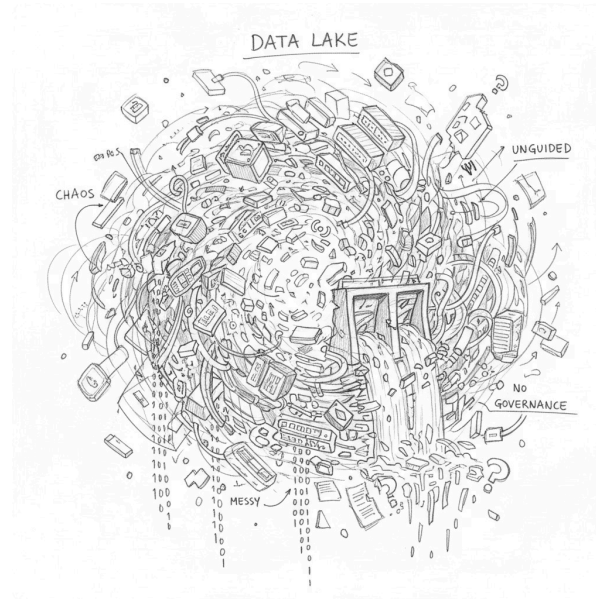
Understanding the unified platform before we write a single line of code.

Current Data World – Challenges



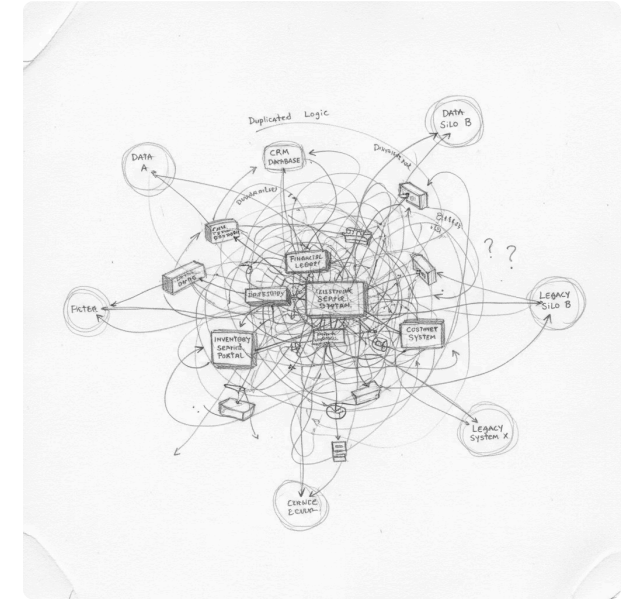
Classic Data Warehouses

- High licensing and infrastructure costs
- Limited scalability
- Rigid schemas
- Mainly for structured data



Data Lake

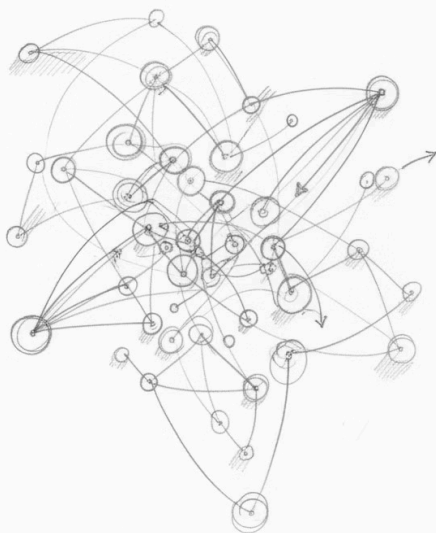
- Cheap storage, but "data swamp"
- Lack of ACID transactions (Atomicity, Consistency, Isolation, Durability)
- Management issues (Data Governance)
- Difficult data discovery and lineage



Distributed Environments

- Separate systems for Business Analytics and Machine Learning
- Data and logic duplication
- Organizational silos
- Performance issues at scale

Impact of Artificial Intelligence and Large Language Models on Data Platform Requirements



→ **Unstructured Data**

AI models require access to text, images, audio, and video. Much more than traditional tables.

→ **Unified Platform**

Need for a single place for raw data, features, machine learning models, and metadata. Eliminating copying between systems.

→ **Governance and Quality**

AI/LLMs multiply data quality issues. Increased importance of data lineage, auditing, and access control.

→ **Data Warehouse + Data Lake + Machine Learning Convergence**

The necessity of a platform combining data warehouse capabilities, data lake flexibility, and AI support in one solution.

The era of generative AI fundamentally changes expectations for data platforms, forcing consolidation and modernization of architecture.

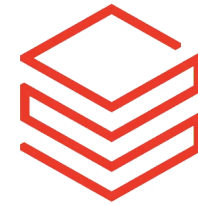
Databricks: Data + AI Platform

What is Databricks?

Databricks is a unified analytics platform built on Apache Spark, created by the project's founders from UC Berkeley. It provides a comprehensive lakehouse solution, combining:

- Data engineering – data processing and preparation
- Data science – advanced analytics and machine learning
- Business analytics – visualizations and business reports

The platform is available across all major public clouds (AWS, Azure, GCP), allowing for flexible deployments consistent with existing infrastructure.



databricks



Founding of Databricks

2013: Founding of Databricks

The founders – creators of Apache Spark from UC Berkeley, including Matei Zaharia, Ali Ghodsi, Ion Stoica – decide to commercialize the technology.

Company Mission

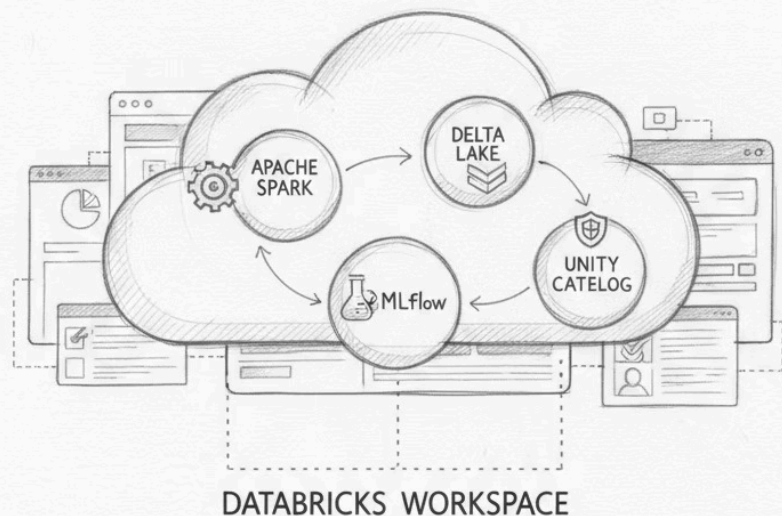
Simplifying the use of Apache Spark in production and cloud environments.
Making advanced analytics accessible to a wider range of organizations.

Databricks was founded by a team that not only created Spark, but continues to actively develop both the engine itself and the ecosystem around it.

Founding team

What is Databricks?

Databricks is a cloud-based Data + AI platform built on proven open-source technologies:



Apache Spark

Distributed in-memory data processing, a standard analytical engine for large datasets.

Delta Lake

Open table format with ACID transactions and time-travel capabilities, the foundation of Lakehouse.

MLflow

Open platform for managing the machine learning (ML) model lifecycle, from experimentation to deployment.

Unity Catalog

Central metadata catalog with advanced governance, data lineage tracking, and access control.

Available on major public clouds, including Azure Databricks – natively integrated with the Microsoft Azure ecosystem.

Databricks Architecture: Control Plane vs. Data Plane

Control Plane

Managed by Databricks:

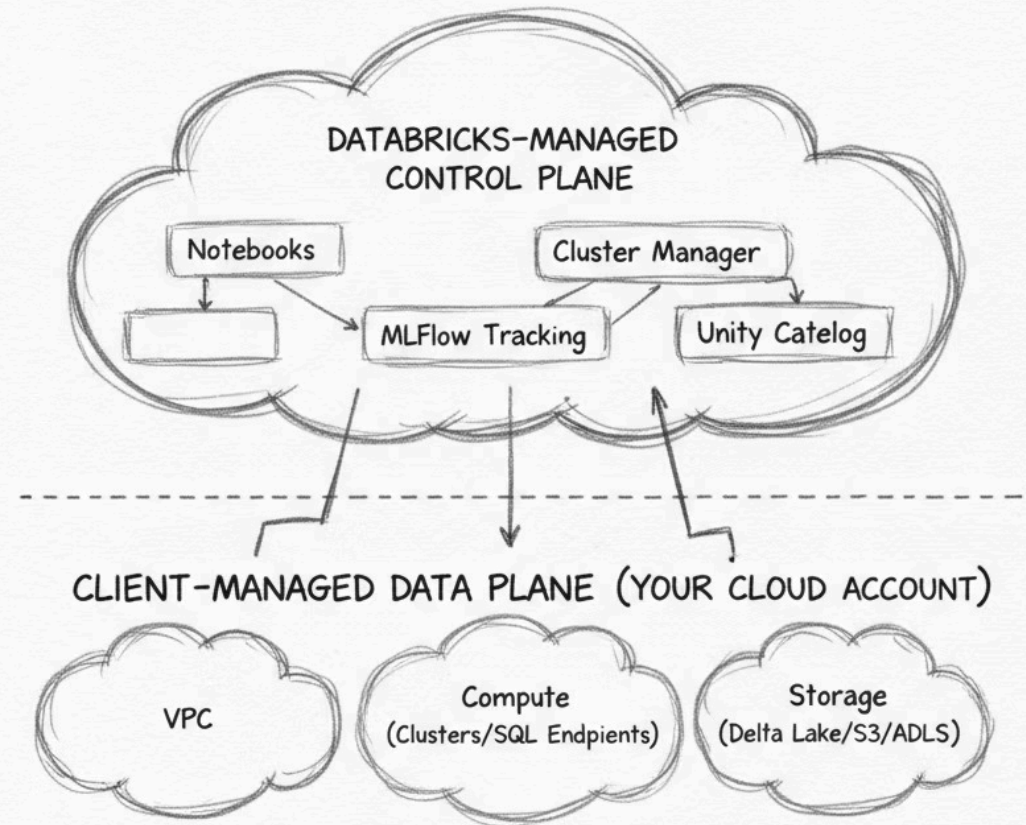
- Web application – user interface
- Cluster manager – cluster management
- Job scheduler – job scheduling
- Notebook server – development environment
- Security and Governance – security
- Metastore – metadata

Data Plane

Client environment:

- Compute clusters (Apache Spark)
- Client storage (S3/ADLS/GCS)
- Network (VPC/VNet)
- Security groups and policies

Communication between layers occurs via secure API calls, ensuring client data isolation.



High-level architecture

Databricks Lakehouse

Storage Layer

Object storage (ADLS/S3/GCS) – cheap, scalable, durable. All data in one place without duplication.

Tables Layer

Delta Tables, Unity Catalog, metadata. Abstraction layer providing structure and transactionality.

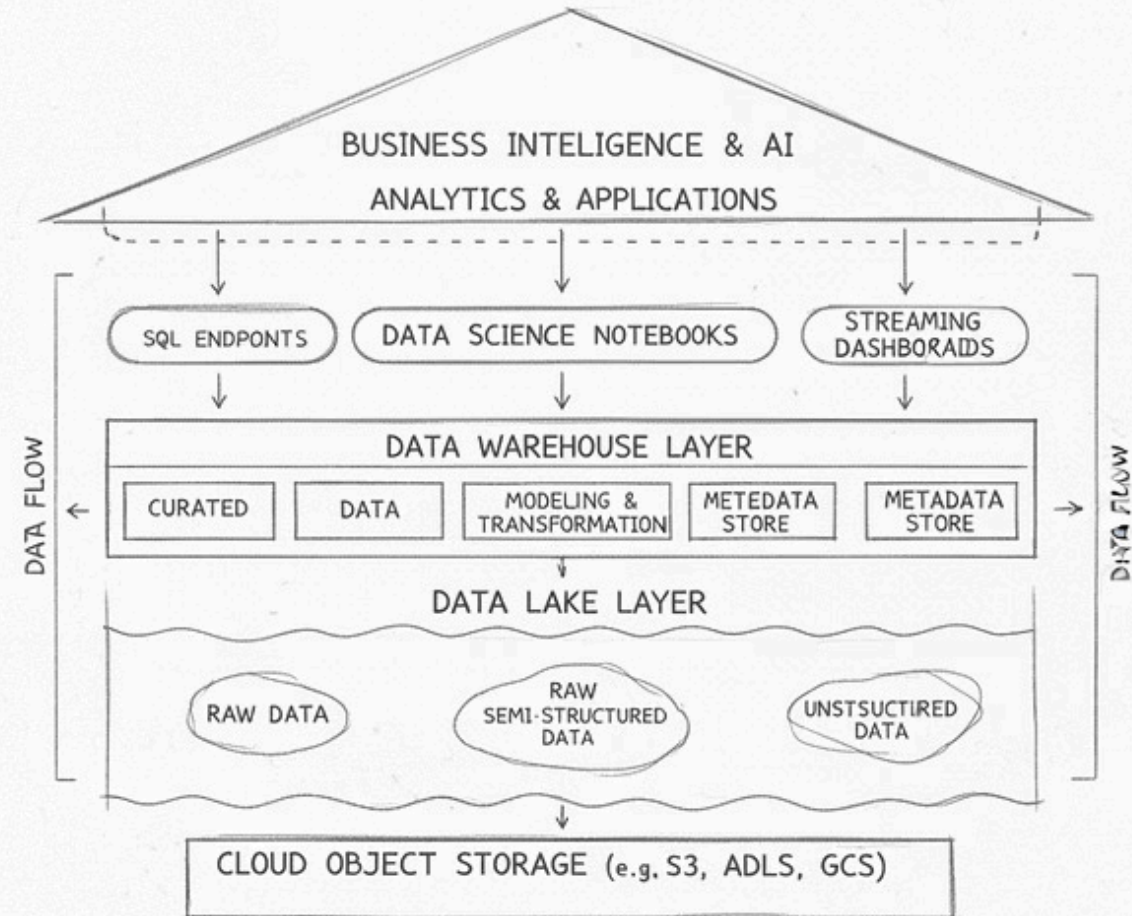
Compute Layer

Apache Spark clusters, SQL Warehouses with Photon. Distributed compute engine scaling flexibly.

Governance Layer

Unity Catalog, security policies, data lineage. Uniform access management and audit for the entire platform.

LAKEHOUSE ARCHITECTURE



Databricks Cloud Integration

Data Storage

Native integration with Azure Data Lake Storage, AWS S3, Google Cloud Storage. Databricks reads and writes data directly to the cloud, without intermediaries.

Security and Identity

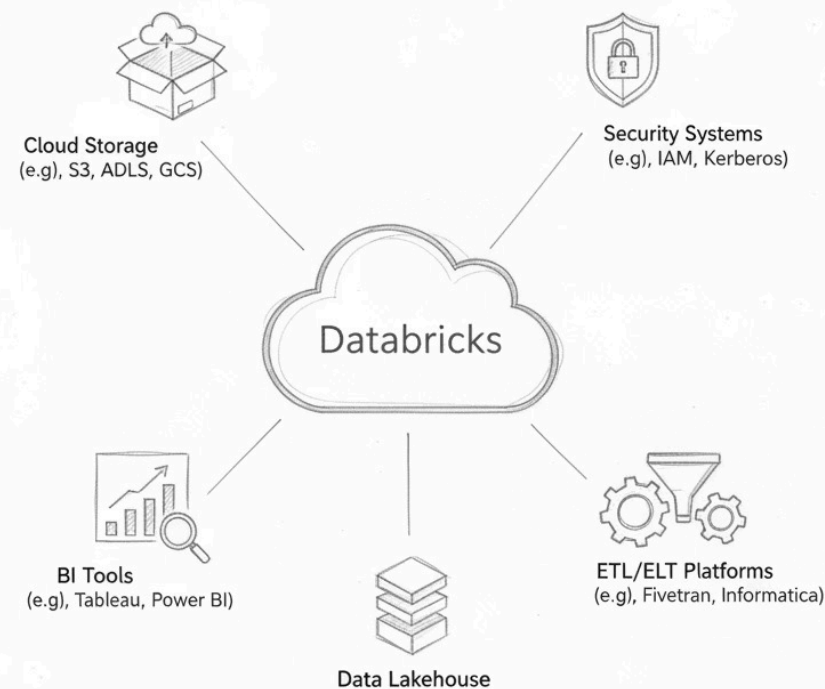
Azure Active Directory, AWS IAM, Google Cloud IAM for authentication. SCIM provisioning and SSO for enterprises. Encryption at rest and in transit.

BI Integrations

Connectors for Power BI, Tableau, Looker, Qlik. JDBC/ODBC drivers for any SQL tools. Partner Connect for quick setup.

ETL/ELT Integrations

Collaboration with Fivetran, dbt, Azure Data Factory, Informatica, Talend. Auto Loader for incremental import of files from cloud storage.



Core Components of the Platform



Workspace

Your working environment: notebooks, Git repositories, pipelines, and jobs — all in one collaborative space.



Compute

Spark clusters managed by Databricks. You pay per DBU × time. Multiple cluster types for different workloads.



Storage

Data lives in your cloud storage (ADLS, S3, GCS). Databricks does not store your data — it only processes it.



Unity Catalog

A central metadata catalog and governance layer for the entire organization — tables, permissions, lineage, audits.



Delta Lake

A transactional layer on top of Parquet files. The default table format in Databricks — brings ACID to your data lake.

Clusters — Types and Best Practices

All-Purpose Cluster

Interactive notebook work. Shared between users. Flexible but more expensive — auto-terminate after 10 min idle to avoid waste.

Job Cluster

Created automatically for a job run, deleted when complete. Cheaper and isolated — the right choice for production pipelines.

SQL Warehouse

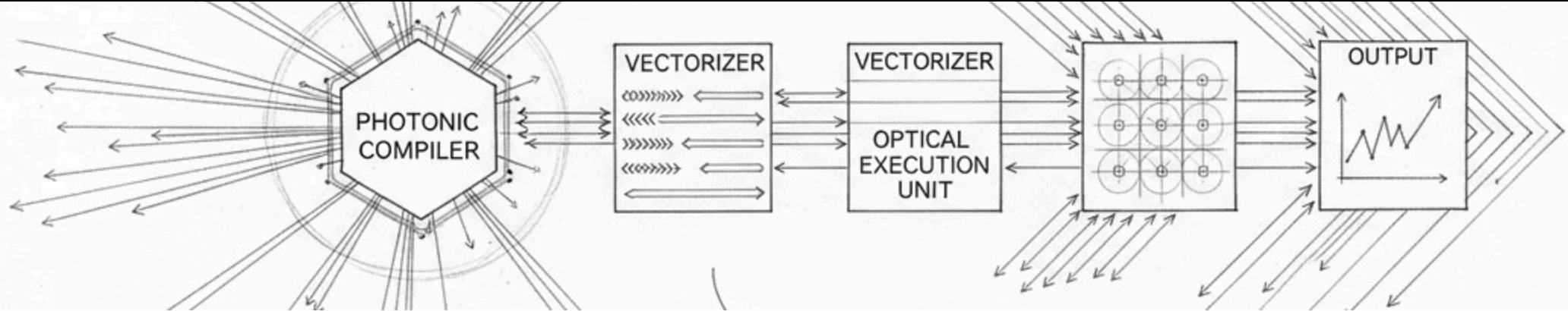
Dedicated compute for SQL Analytics and dashboards. Optimized for BI queries and concurrent user access.

Autoscaling

The cluster automatically adjusts the number of worker nodes based on the current workload — scale up when busy, scale down when idle.

DBU — Databricks Unit

The billing unit for Databricks compute. Price depends on the cluster type and cloud provider. Monitor usage in the Account Console.



What is Photon

Photon is a high-performance, native query engine created by Databricks:

Vectorized Execution

Operations on data columns instead of row by row. SIMD instructions for maximum CPU performance.

Compatibility

Full compatibility with Spark SQL and DataFrame API. No code changes needed – enable Photon with a single switch.

Business Goal

Significantly faster analytical queries at a lower compute cost. Better TCO for SQL and BI workloads.

SECTION 3

Apache Spark Architecture

Understanding how Spark executes your code is the key to writing efficient, scalable data pipelines.

Spark Execution Model — Driver & Executors

Driver

A single master node that compiles your code, builds the DAG of operations, and coordinates task distribution to workers. All control logic runs here.

Warning

Never call `.collect()` on a large DataFrame — it loads everything into Driver memory and can crash the node.

Executors

One to N worker nodes that execute tasks and store intermediate data in memory. Each task processes exactly one partition of data.

Partitions

The fundamental unit of parallelism in Spark. More partitions = more parallel tasks. Data never leaves the cluster until an action is triggered.

Shuffle

The most expensive operation in a pipeline — moves data between partitions across the network. Minimize shuffles for best performance.

Lazy Evaluation — Transformations vs. Actions

Transformations (Lazy)

Build the execution plan — **no computation happens**.

Examples: `select`, `filter`, `withColumn`, `join`, `groupBy`. Spark records what to do, not when.



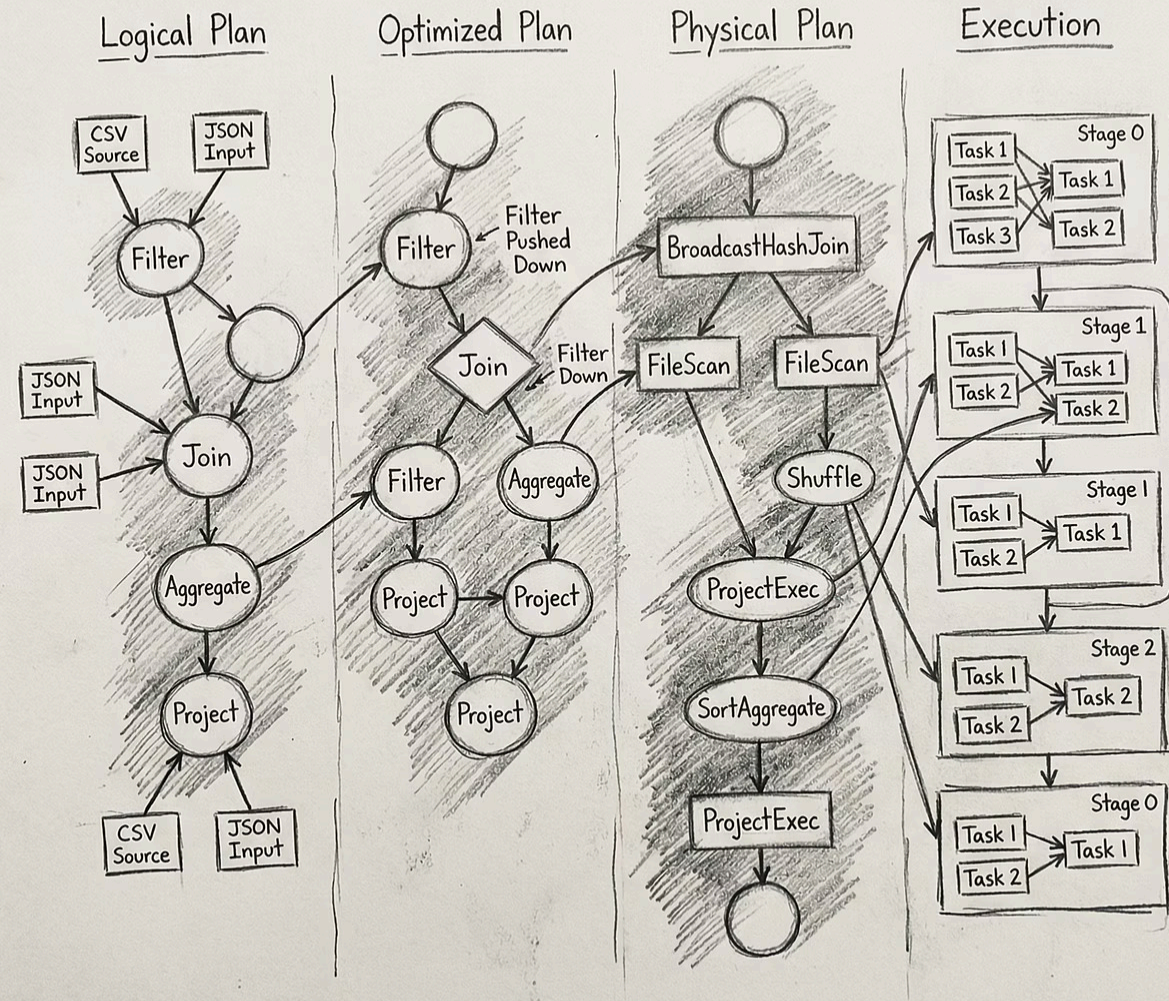
Actions (Eager)

Trigger the execution of the entire plan. Examples: `count`, `show`, `display`, `write`, `collect`. This is when Spark actually touches the data.

- 📌 **Key benefit:** Before executing, Spark's **Catalyst Optimizer** analyzes the full chain of transformations and rewrites it into the most efficient physical plan — reordering filters, eliminating unnecessary columns, and pushing predicates as close to the source as possible.

DAG and the Execution Plan

Every Spark job is represented as a Directed Acyclic Graph (DAG) — a sequence of steps from data source to result. The Catalyst Optimizer applies two key optimizations: **predicate pushdown** (move filters close to the source) and **column pruning** (read only needed columns from Parquet). Use **Spark UI** tabs (Jobs, Stages, SQL) to identify bottlenecks in your pipeline.



PySpark DataFrame API — Reading the Code

DataFrame Model

A distributed table with an immutable schema. Operations are chained: `df.filter(...).withColumn(...).groupBy(...).agg(...)`

Column References

Use `col("name")` for standalone expressions. Use `df["name"]` to avoid ambiguity in joins between two DataFrames with the same column name.

display() vs show()

`display(df)` — Databricks-specific; renders a paginated table in the notebook. `df.show(n, truncate=False)` — standard Spark API; works outside Databricks too.

SQL Inline

`spark.sql("SELECT ...")` runs SQL and returns a DataFrame — you can chain further transformations on the result.

SECTION 4

Ingestion & Medallion Architecture

Organizing your data lake into structured quality layers is the foundation of reliable, auditable pipelines.

Medallion Architecture — Why Three Layers?

🥇 Gold

Business-ready aggregations. One row = one business dimension. Consumed by reports, dashboards, and analysts.

🥈 Silver

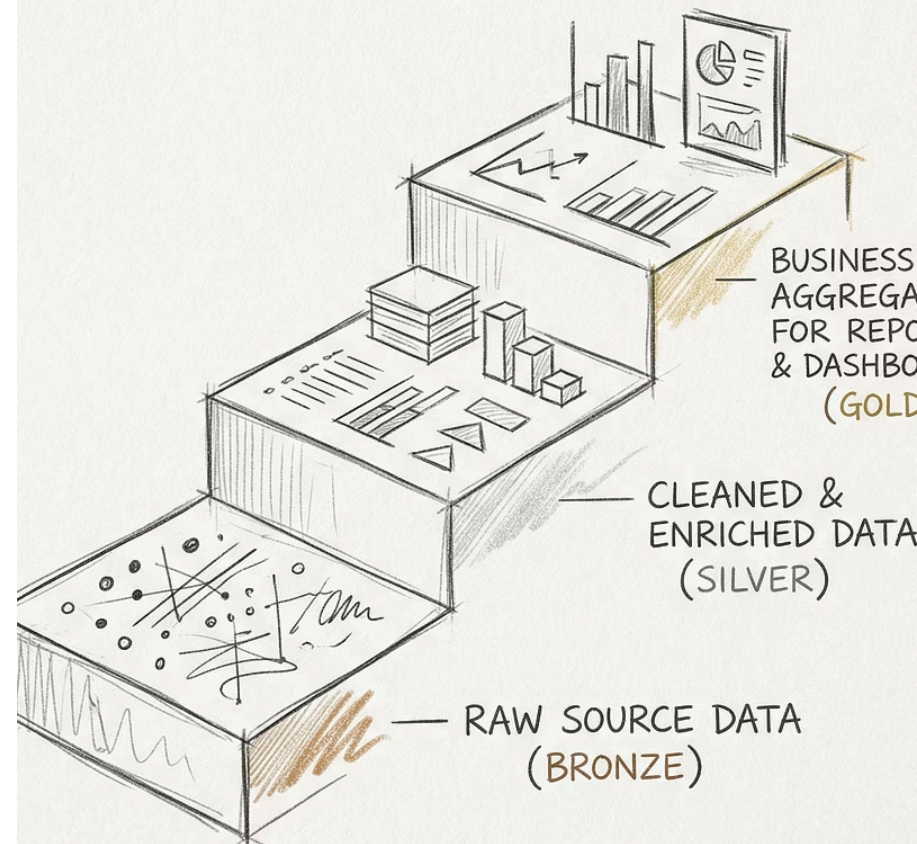
Cleaned, typed, and enriched data. Quality guaranteed. Ready for analytical transformations.

🥉 Bronze

Raw data as received from the source. No filtering, no business logic. The immutable record of truth.

Each layer is a separate Delta table — independently queryable, auditable, and restorable. A failure in Silver never corrupts Bronze. You can always recompute downstream layers from the raw source.

DATA REFINEMENT STAGES



Bronze Layer — Principles

The Golden Rule

Raw as-is: Load exactly what arrived from the source. No filtering, no business logic, no type interpretation at ingestion time.

All Columns = StringType

JSON and CSV have no native types. Bronze preserves the raw strings — your code assigns types in Silver, where you control the logic.

Mandatory Metadata Columns

- `_load_ts` — `current_timestamp()` — when the file was loaded
- `_source_file` — `input_file_name()` — which file the row came from

Always Use Explicit Schema

Never use `inferSchema` in production. It causes a double scan of the data and provides no guarantees. Define a `StructType` explicitly.

Target Format

Delta — for ACID guarantees, schema enforcement, and time travel from day one.

Explicit Schema — StructType & StructField

```
from pyspark.sql.types import StructType, StructField, StringType

schema = StructType([
    StructField("order_id", StringType(), nullable=False),
    StructField("customer_id", StringType(), nullable=True),
    StructField("total_amount", StringType(), nullable=True),
    StructField("order_datetime", StringType(), nullable=True),
    # ... remaining fields
])
```

nullable=False on Keys

Catches missing primary key values at read time — fail fast rather than silently propagating nulls through the pipeline.

Why All StringType in Bronze?

Source formats like JSON and CSV carry no type information. Strings are the safe default — Silver is where you cast to the target types with full control.

Avoid inferSchema

Spark scans the data twice and may guess wrong. In production, guessed types become technical debt that breaks pipelines silently.

Silver Layer — Cleaning & Typing

01

Recast Types

Convert StringType to target types: TimestampType, DoubleType, IntegerType. Always use explicit format strings:
`to_timestamp(col("order_datetime"), "yyyy-MM-dd'T'HH:mm:ss")`

03

Filter Invalid Rows

Remove rows with nulls on key fields. A Silver row is a promise of quality — downstream code should not need defensive null checks.

02

Enrich with Derived Columns

Add business-relevant computed columns: `net_amount` (total after discount), `order_tier` (value segment). These derivations belong in Silver, not Gold.

04

Drop Technical Metadata

Remove columns like `_source_file` that were needed in Bronze but are noise downstream. Keep Silver clean and business-focused.

📝 Write mode: `mode("overwrite") + option("overwriteSchema", "true")` when schema changes are intentional.

Common PySpark Transformations — Quick Reference

Operation	Code
Type cast	<code>col("x").cast(DoubleType())</code>
Parse timestamp	<code>to_timestamp(col("x"), "yyyy-MM-dd'T'HH:mm:ss")</code>
Conditional value	<code>when(condition, value).otherwise(other)</code>
Round number	<code>round(col("x"), 2)</code>
Literal constant	<code>lit(0)</code>
Extract date	<code>to_date(col("ts"))</code>
Filter nulls	<code>col("x").isNotNull()</code>
Drop column	<code>.drop("col_name")</code>

Gold Layer — Business Aggregations

One Row = One Business Dimension

Gold tables are designed for direct consumption. A single row represents a complete summary for one day, one category, or one region — not raw transactions.

Always Rebuilt from Silver

Use `mode("overwrite")` without `overwriteSchema` in Gold. Schema must be stable between runs — reports depend on it.

Typical Aggregation Pattern

```
df_gold = df_silver.groupBy("order_date").agg(
    count("*").alias("order_count"),
    sum("net_amount").alias("total_revenue"),
    round(avg("net_amount"), 2)
    .alias("avg_order_value")
)
```

Use `net_amount`, Not `total_amount`

Always aggregate the post-discount amount. `total_amount` is the pre-discount figure from Bronze — using it in Gold would inflate revenue metrics.

SECTION 5

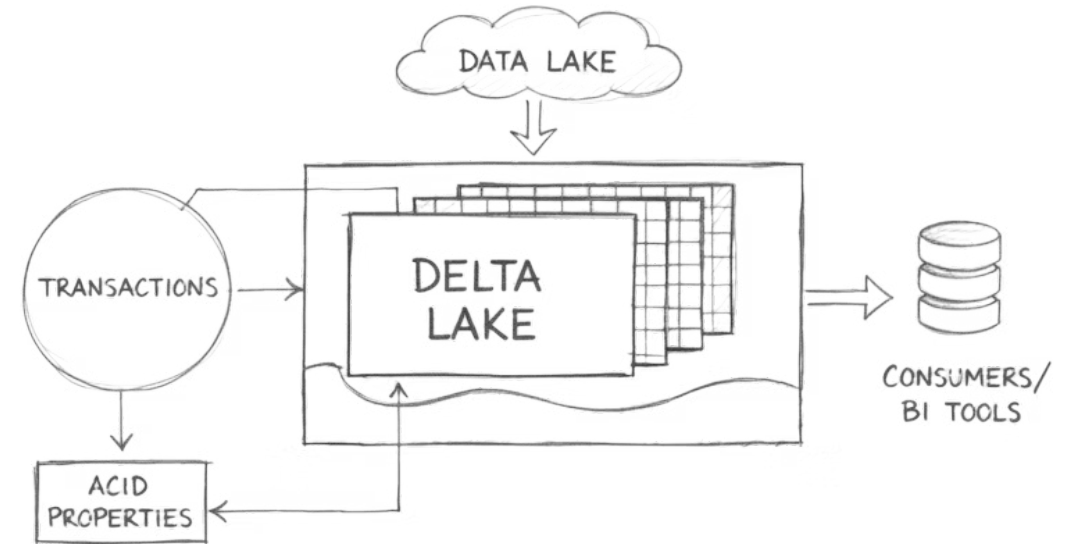
Delta Lake

Delta Lake transforms a plain file storage into a reliable, transactional data platform.

What is Delta Lake?

Delta Lake is an open-source storage layer that brings reliability to Data Lakes. It combines the best features of traditional databases and Data Lakes, providing:

- **ACID Transactions:** Ensures atomicity, consistency, isolation, and durability of operations, guaranteeing data integrity.
- **Schema Enforcement:** Helps maintain data quality by blocking writes that do not conform to the table's schema.
- **Data Versioning:** Allows access to historical versions of tables, facilitating auditing, rolling back changes, and data recovery.
- **Scalable Metadata:** Supports tables with billions of partitions and files.
- **Unified Streaming and Batch Processing:** Allows the use of the same Data Lake for real-time and batch data processing.



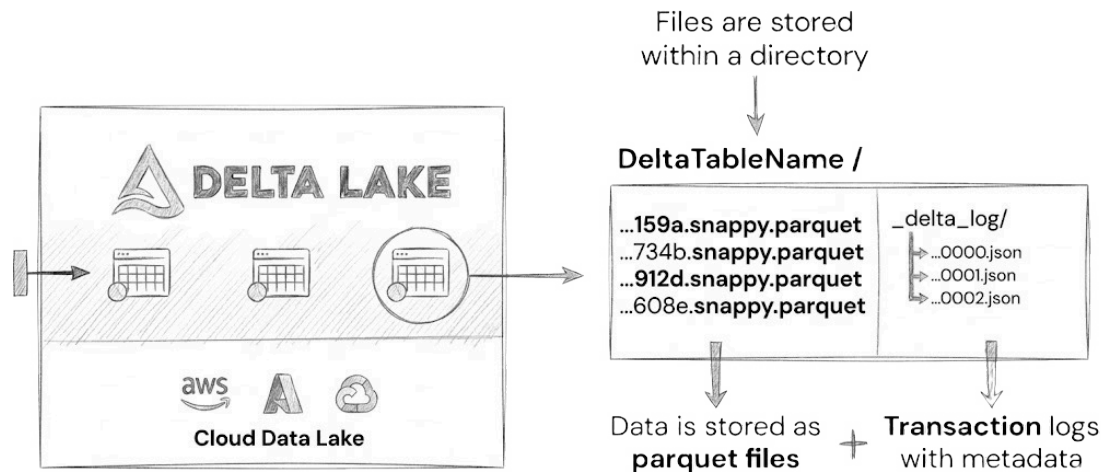
Delta Log – The Heart of Transactionality

The Delta Log is an ordered sequence of JSON files stored in the directory `_delta_log`. Each file represents a single transaction and contains complete information about changes in the table.

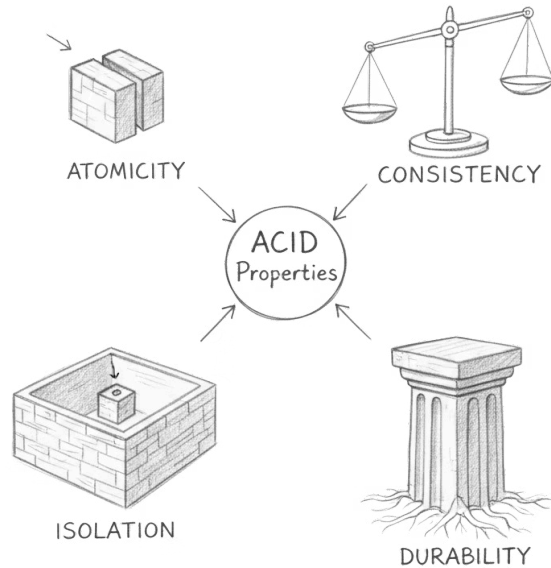
Structure of a Delta Log entry:

- List of added Parquet files with metadata
- List of deleted files
- Schema operations (adding/removing columns)
- Operation metadata (timestamp, user, operation)
- Data statistics (min/max values for data skipping)

Each read operation begins by reading the Delta Log to determine the current state of the table and the list of active files.



ACID Properties in Data Lake



Atomicity

All operations are atomic – either complete success or complete rollback. No partial writes or inconsistent states.

Consistency

Schema enforcement ensures that each transaction leaves the table in a valid state consistent with the defined schema.

Isolation

Serializable isolation – concurrent operations do not interfere with each other. Readers always see a consistent snapshot.

Durability

Committed transactions are durable and survive system failures thanks to the Delta Log stored in object storage.

DESCRIBE HISTORY – Transaction audit

```
DESCRIBE HISTORY bronze_transactions;
```

-- The result shows the full history of operations:

version	timestamp	operation	operationParameters
0	2024-01-15 10:00:00	CREATE	{mode: ErrorIfExists}
1	2024-01-15 11:30:00	WRITE	{mode: Append}
2	2024-01-15 14:00:00	MERGE	{predicate: ["id"]}
3	2024-01-15 16:45:00	DELETE	{predicate: ["status='invalid'"]}
4	2024-01-15 18:00:00	OPTIMIZE	{predicate: []}

The `DESCRIBE HISTORY` command reveals the full audit trail of all operations performed on a Delta table. Each version is an immutable snapshot that can be used for time travel, forensic analysis, or disaster recovery.

The transaction history is automatically managed by Delta Lake and stored according to the retention period configuration (default 30 days).

Schema Enforcement vs. Schema Evolution

Schema Enforcement (Default: ON)

Writing data with a schema that doesn't match the table throws an `AnalysisException`. This protects data integrity in production — bad data fails loudly rather than silently corrupting the table.

Use in: All production pipelines.

Schema Evolution

Add `option("mergeSchema", "true")` to allow new columns to be automatically added to the table schema. Existing columns are not changed.

Use in: Migrations, gradual field additions, development.

`overwriteSchema`

Used with `mode("overwrite")` — completely replaces the schema. Use consciously — this is a breaking change for any downstream consumer.

Time Travel

Time travel through data history

Time Travel – Historical Read

Historical Query Syntax

```
-- Read by version number
SELECT * FROM table_name VERSION AS OF 0;

-- Read by timestamp
SELECT * FROM table_name
TIMESTAMP AS OF '2024-01-15 10:00:00';

-- In DataFrames API
df = spark.read \
    .format("delta") \
    .option("versionAsOf", 0) \
    .load(path)
```

Time Travel is a unique feature of Delta Lake that allows reading data from any point in the table's history. Each table version is an immutable snapshot that remains available according to the retention policy.

Business Use Cases

- Regulatory audit and compliance
- Analysis of data changes over time
- Recreating past reports
- Comparing before/after versions
- Testing transformation logic
- Data recovery after erroneous operations

❏ Time Travel works without additional storage costs — it uses the same Parquet files as the current version

Time Travel – Practical Applications

Time Travel functionality in Delta Lake is widely used in daily data operations, enabling not only auditing but also quick reaction to errors and trend analysis.

Use Cases:

Auditing data changes

Restoring state after erroneous operations

Comparing data states between versions

Debugging data pipelines

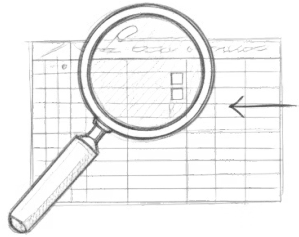
Limitations:

- Performance impact with very old versions
- Costs of storing historical versions
- Retention period (30 days by default)

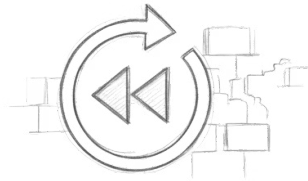
Practical Examples:

```
-- Comparing data between versions
SELECT COUNT(*) as current_count FROM
sales_table;
SELECT COUNT(*) as yesterday_count
FROM sales_table TIMESTAMP AS OF '2024-
01-14';
```

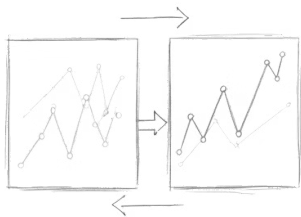
```
-- Finding differences
SELECT * FROM sales_table
EXCEPT
SELECT * FROM sales_table VERSION AS OF
1;
```



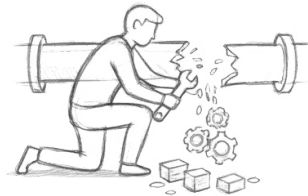
DATA AUDITING



ROLLBACKS



VERSION COMPARISONS



DATA PIPELINE
DEBUGGING

Time Travel – Practical Applications

Time Travel functionality in Delta Lake is widely used in daily data operations, enabling not only auditing, but also quick response to errors and trend analysis.

Use Cases:

Auditing data changes

Restoring state after erroneous operations

Comparing data states between versions

Debugging data pipelines

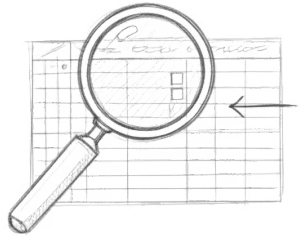
Limitations:

- Performance impact with very old versions
- Costs of storing historical versions
- Retention period (30 days by default)

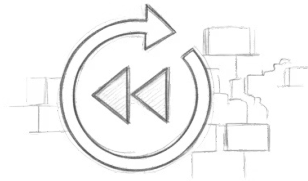
Practical Examples:

```
-- Compare data between versions
SELECT COUNT(*) as current_count FROM
sales_table;
SELECT COUNT(*) as yesterday_count
FROM sales_table TIMESTAMP AS OF '2024-
01-14';
```

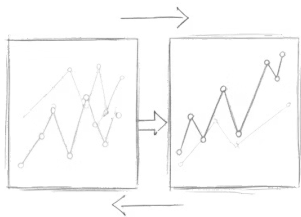
```
-- Find differences
SELECT * FROM sales_table
EXCEPT
SELECT * FROM sales_table VERSION AS OF
1;
```



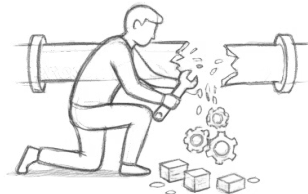
DATA AUDITING



ROLLBACKS



VERSION COMPARISONS



DATA PIPELINE
DEBUGGING

RESTORE — Undoing Changes

-- Restore to a specific version

RESTORE TABLE orders TO VERSION AS OF 2

-- Restore to a specific point in time

RESTORE TABLE orders TO TIMESTAMP AS OF '2024-01-10 08:00:00'

RESTORE Creates a New Version

It does not delete history. The RESTORE operation itself is recorded as a new version — fully auditable. Your table history remains intact.

Faster Than Backup Restore

For accidental DELETES or OVERWRITES, a RESTORE command is orders of magnitude faster than restoring from a traditional backup system.

Retention Constraint

You can only restore to versions whose data files still exist. If VACUUM has removed old files, those versions are no longer restorable.

SECTION 6

Orchestration & Workflows

Moving from manual notebook runs to reliable, monitored, scheduled production pipelines.

Why Do We Need Orchestration?

The Problem: Manual Runs

Running notebooks manually offers no repeatability, no audit trail, no automated alerting, and no dependency management. One missed run = stale data in production.

The Solution: Workflow Jobs

A Workflow Job is a declared DAG of tasks with a schedule and monitoring. It handles retries, dependencies, repair runs, and cost tracking automatically.

vs. Classic Cron

Databricks Workflows add task dependencies, parallel execution, per-task retry, and repair run — features a cron job simply cannot provide.

Job Cluster Economics

Created automatically before the run, deleted after completion. No idle compute costs. Production pipelines should always use Job Clusters.

Full Auditability

Every run is logged with start time, end time, cost, status, and output. You can reproduce exactly what ran and when.

Anatomy of a Workflow Job



Task

The atomic unit of work: a notebook, Python script, SQL statement, or Delta Live Tables pipeline. Each task has its own configuration and status.



Depends On

The DAG edge. The Silver task only starts after the Bronze task succeeds. Failures stop the cascade — downstream tasks are skipped, not corrupted.



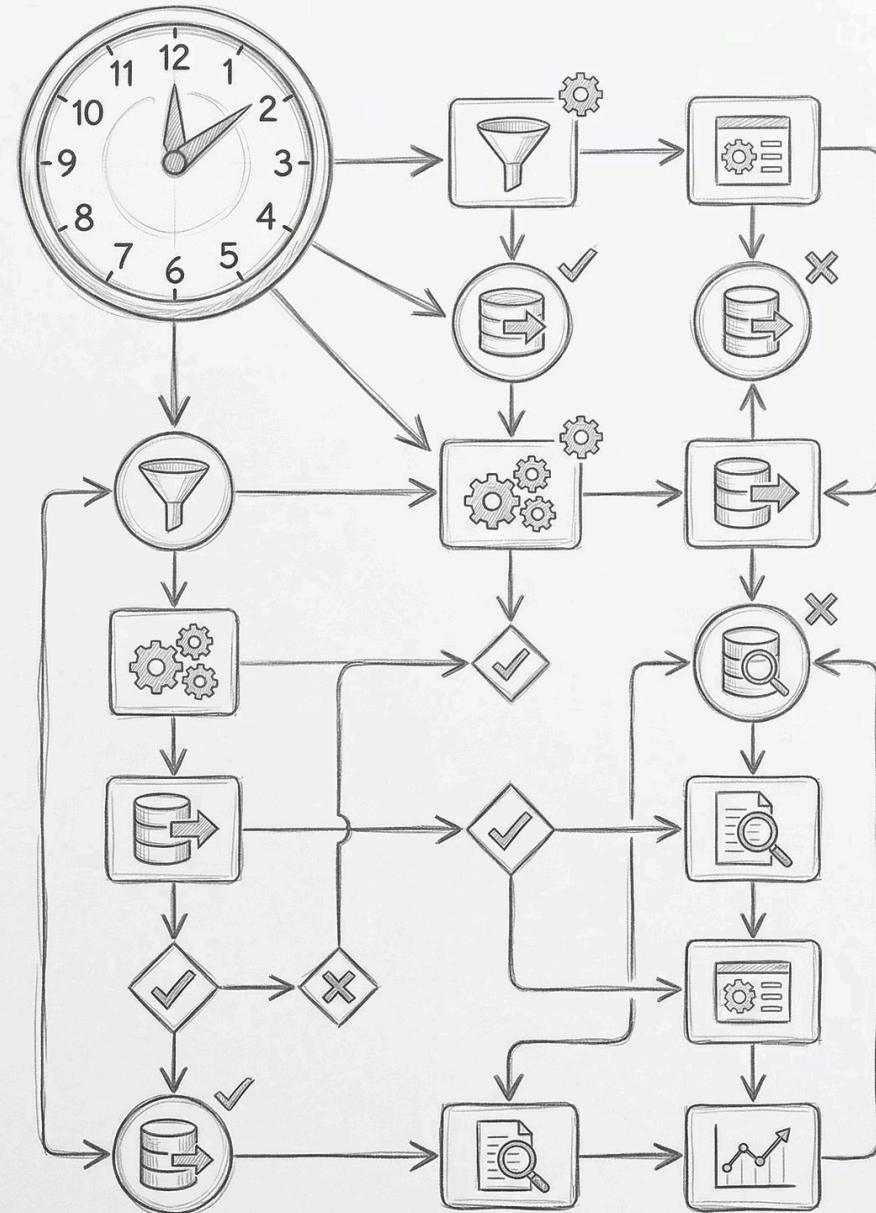
Parameters

Passed to notebooks via `dbutils.widgets` or `sys.argv`. Makes pipelines reusable for different dates, environments, or data sources.



Retries

Automatic retry on failure. Configurable per task — critical tasks can retry 3 times, while idempotent tasks may only retry once.



Scheduling & Monitoring

CRON Quartz Syntax

```
0 0 6 * * ?
```

= every day at 06:00

Trigger Types

- **Manual** — run on demand
- **CRON** — time-based schedule
- **File Arrival** — trigger on new data
- **Continuous** — streaming jobs

Repair Run

Re-runs **only the failed tasks** in a job — not the entire pipeline. If your Bronze task succeeds but Silver fails, Repair Run starts from Silver. Saves time and compute cost.

Alerts

Email or webhook notifications on: Job Start, Success, Failure, Skipped. Configure per-job or per-task for fine-grained alerting.

Metrics Panel

Per-run execution time, cost, and history of the last N runs — available directly in the Workflows UI.

SECTION 7

Unity Catalog & AI/BI

Enterprise governance, data lineage, and built-in analytics — all within Databricks.

Unity Catalog — Governance for the Entire Organization

Single Catalog for All Workspaces

Before Unity Catalog, each workspace had its own Hive metastore — creating data silos. UC provides one unified catalog across your entire Databricks account.

Data Lineage

Automatic tracking of data flow between tables — including column-level lineage. Know exactly where each field comes from and where it goes.

Audit Log

Every SELECT, INSERT, and GRANT is recorded — who, when, what. Essential for compliance, security investigations, and GDPR auditing.

Tags & Comments

Document tables and columns directly in the catalog — no separate wiki needed. Tags enable discovery and classification at scale.

Permissions Model

```
GRANT SELECT ON TABLE catalog.schema.orders TO `analyst@company.com`;  
GRANT USE SCHEMA ON SCHEMA catalog.bronze TO `data_engineers`;  
GRANT ALL PRIVILEGES ON CATALOG dev TO `admin_group`;
```

Privilege Hierarchy

Permissions cascade: `USE CATALOG` → `USE SCHEMA` → `SELECT` → `MODIFY` → `CREATE TABLE` → `ALL PRIVILEGES`. Grant at the right level to minimize maintenance.

Row & Column Security

Filter data at query time without creating separate views. Users only see the rows and columns they are authorized for — enforced by the catalog.

Service Principals

Automated jobs must never use a personal account. Use service principals — dedicated identities for pipelines with minimal required privileges.

AI/BI Dashboards

Native Dashboards in Databricks

Build and share dashboards directly in the platform — no export to Tableau or Power BI required. Source any Delta table or SQL result set.

Refresh Options

- On-demand manual refresh
- Scheduled refresh via CRON
- Live query on dashboard open

Chart Types

Bar, line, scatter, pie, counter, heatmap, and map — covering most BI use cases without leaving Databricks.

Genie — AI Assistant

Users ask questions in natural language. Genie generates SQL, executes it, and presents the result as a chart — no SQL knowledge required for end users.

Sharing Dashboards

Share via link. In the right configuration, recipients do not need a Databricks account — making it practical for business stakeholders who are not platform users.

SECTION 8

Final Project: RetailHub Pipeline

Putting everything together — build a complete, production-grade pipeline from raw orders to a daily business report.

Business Scenario

The Company

RetailHub processes customer orders from multiple sales channels. Orders arrive as JSON batch files — one file per day.

The Goal

Every morning at **06:00**, automatically refresh the daily revenue report in the Gold table — ready for analysts when they start the day.

Input Data

`orders_batch.json` — raw orders for one day. Contains order IDs, customer IDs, amounts, discounts, timestamps, and channel information.

Expected Output

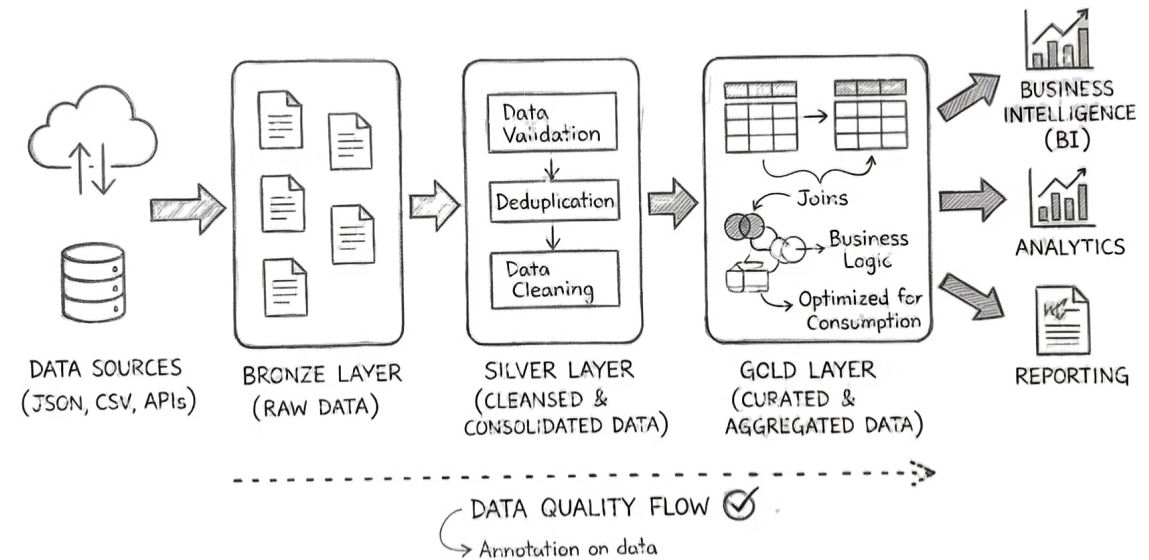
Table `gold.lab_daily_orders` — one row per day, four columns: date, order count, total revenue, and average order value.

Constraint

The pipeline must be idempotent — running it twice for the same day produces the same result. No duplicates in Gold.

Pipeline Architecture

Each stage is implemented as a separate notebook and Delta table. The pipeline is orchestrated by a Databricks Workflow Job with explicit task dependencies — Bronze must complete before Silver starts, Silver before Gold.



Gold Table Schema

Column	Type	Description
order_date	DateType	Aggregation day — one row per date
order_count	LongType	Number of orders on this day
total_revenue	DoubleType	Sum of net_amount (after discount)
avg_order_value	DoubleType	Average order value, rounded to 2 decimal places

- ❏ The Gold schema is intentionally minimal and stable. Reports and dashboards depend on this structure not changing between runs. Any schema change is a breaking change — coordinate with consumers before modifying.

Verification After Running the Job

-- Did data land in the table?

```
SELECT * FROM gold.lab_daily_orders  
ORDER BY order_date DESC  
LIMIT 10;
```

-- How many days are covered?

```
SELECT COUNT(*) AS total_days  
FROM gold.lab_daily_orders;
```

-- Revenue sanity check

```
SELECT  
  SUM(total_revenue),  
  MIN(order_date),  
  MAX(order_date)  
FROM gold.lab_daily_orders;
```

-- Audit: what operations were performed?

```
DESCRIBE HISTORY gold.lab_daily_orders;
```

These four queries form a standard post-run verification checklist. Run them after every pipeline execution during development — and automate them as part of your data quality checks in production.

SECTION 9

Wrapping Up

Reviewing what we built today and charting the path forward.

What We Built Today

Layer	Notebook	Key Technique
Bronze	lab_task_01_bronze	StructType, StringType, metadata columns
Silver	lab_task_02_silver	cast, to_timestamp, when, filter nulls
Gold	lab_task_03_gold	groupBy, agg, to_date, round
Orchestration	WORKSHOP_final_project	Workflow Job, CRON, notebook.exit

In one day, you went from raw JSON files to a fully orchestrated, scheduled, production-style data pipeline — complete with ACID guarantees, time travel, and automated daily refresh. This is exactly the pattern used in real enterprise Lakehouse deployments.

Key Concepts to Remember

Lakehouse

Don't choose between a Data Warehouse and a Data Lake — Databricks gives you both. Flexible storage with enterprise-grade transactions and governance.

Medallion

Three layers = three quality contracts. Bronze is a promise of raw completeness. Silver is a promise of cleanliness. Gold is a promise of business accuracy.

Delta

ACID and time travel are not optional extras — they are the standard. Never use plain Parquet for tables that will be updated or audited.

Explicit Schema

inferred schema is technical debt. Define StructType upfront — it protects your pipeline from source changes and makes failures visible immediately.

Job Cluster

Production always runs on a Job Cluster. All-purpose clusters are for interactive development only — never leave them running when not in use.

Unity Catalog

Governance at the column level, not just the table level. One catalog, one permission model, full lineage — for the entire organization.

Q&A and Feedback

Open Questions

Now is the time for any questions that came up during the day — about the platform, the pipeline, the architecture, or anything else from the training.

No question is too basic. If you're wondering about it, someone else in the room probably is too.

Feedback Survey

Please take 2 minutes to fill in the feedback form — your input directly shapes future iterations of this training.

The link or QR code will be shared on screen now.

Stay in Touch

Reach out to the trainer with follow-up questions after the session. Details will be provided at the end of the survey.

📄 Thank you for your attention, your questions, and your energy throughout the day. Happy engineering! 🚀